# Welcome!

We'll get started shortly.  Please take the Zoom poll in the meanwhile!

# CS 49 Week 8

Surajit A. Bose

# How to get hold of me / get help from other resources

- Surajit's office hours
  - Fridays 12 noon–1p, directly after section
  - By appointment on Zoom
- Post on the section forum, usually get a response within 2 hours
- Email bosesurajit@fhda.edu or Canvas inbox, 24 hr turnaround
- The github repo has section materials, starter code, and solutions

---

- Canvas inbox for Lane
- The main forum for the course on the Code in Place site
- Lane's office hours
- Online or in-person tutoring (Room 3600)
- One-on-one code reviews with Sarah or Lia

# Agenda

- Goal: To understand and learn to use lists in  Python
- Types
  - Atomic vs Container
  - Mutable vs  Immutable
- Lists
  - Creating lists
  - Accessing list elements
  - Useful list methods
- Section  Problem: Fruit  List

# Types of  Python types

# Atomic and Container Types (Slide 1 of 2)

- In Python, **`int`**, **`float`**, **`Boolean`**, or **`None`** are all **`atomic`** types
- A value of those types is a single value
- Python also has **`container`** types
- Containers are what they sound like
  - They can be empty, or they can hold any number of values
  - They are **`collections`** of elements
  - We can access individual elements in the containere
  - We can **`iterate`** (loop) over all the elements in a container
- Surprise! We've been working with one container type already
  - Any guesses as to what?

# Atomic and Container Types (Slide 2 of 2)

- In Python, **`int`**, **`float`**, **`Boolean`**, or **`None`** are all **`atomic`** types
- A value of those types is a single value
- Python also has **`container`** types
- Containers are what they sound like
  - They can be empty, or they can hold any number of values
  - They are **`collections`** of elements
  - We can access individual elements in the containere
  - We can **`iterate`** (loop) over all the elements in a container
- Surprise! We've been working with one container type already
  - **`str`**

# Mutable vs  Immutable Types (Slide 1 of 2)

- All the types we've been looking at so far have been **immutable**
- By definition, immutable objects cannot be changed in memory; they can only be replaced with a different object
- All atomic types are immutable
- Strings are immutable:

```
hi = 'Hello'            # creates a string in memory
                        # and assigns it to the variable hi

hi = 'Hello there'      # does not modify the original string
                        # in memory; creates a new string
                        # and assigns it to the variable hi
```

# Mutable vs  Immutable Types (Slide 2 of 2)

- Python has several **`mutable`** container types
- These containers can be changed in memory without being replaced
- We can add elements to or remove elements from the container
- One useful mutable container type is **`list`**
- As with **`str`**:
  - A **`list`** can be empty or arbitrarily large
  - We can iterate over their elements of a **`list`**
- Unlike **`str`**:
  - We can mutate a **`list`**
  - Elements in a **`list`** do not all have to be the same type

# Lists

# Creating a list

- A new, empty list can be created with a pair of square brackets:

```
my_list = []
```

- A new list can also be created with elements in it:

```
colors = ['red', 'blue']
```

- List elements are separated by commas
- Another way to create a new list is by adding two lists together:

```
more_colors = ['green', 'yellow']
colors_two = colors + more_colors
# colors_two is ['red', 'blue', 'green', 'yellow']
# Original lists colors, more_colors are unchanged
```

# Accessing list elements (Slide 1 of 2)

- An element's position in the list is its index
- Indices start at zero and go up to one less than the number of elements
- E.g., given **`colors_two = ['red', 'blue', 'green', 'yellow']`**
  - **`'red'`** is at index 0
  - **`'blue'`** at index 1
  - **`'green'`** at index 2
  - **`'yellow'`** at index 3
- Access an element with the index in square brackets after the list name

```
shrek_color = colors_two[2]  # shrek_color is 'green'
```

# Accessing list elements (Slide 2 of 2)

- We can also use negative indices, with index of the last element being -1
- E.g., given **`colors_two = ['red', 'blue', 'green', 'yellow']`**
  - **`'yellow'`** is at index -1
  - **`'green'`** at index -2
  - **`'blue'`** at index -3
  - **`'red'`** at index -4
- Access an element with the index in square brackets after the list name

```
gumby_color = colors_two[-3] # gumby_color is 'blue'
```

# Useful list methods (Slide 1 of 2)

- Given `numbers = [79, 84, 20, 5]`
- We can print the entire list:

```
print(numbers)
```

- We can iterate over a list in a for-each loop:

```
for number in numbers:
    print(f"Current number is: {}")
```

- We can add to the end of the list

```
numbers.append(13)          # list is now [79, 84, 20, 5, 13]
```

- We can get the length of the list

```
how_many = len(numbers)     # how_many is 5
```

- Worked example: Index Game.

# Useful list methods (Slide 2 of 2)

- Given `numbers = [79, 84, 20, 5, 13]`
- We can check whether a particular number is in the list:

```
18 in numbers      # True if 18 is in the list, else False
```

- We can get the index of a particular element:

```
my_ind = numbers.index(84)    # my_ind is 1
```

- We can remove from the end of the list

```
my_num = numbers.pop()        # list is now [79, 84, 20, 5]
                              # my_num is 13
```

- We can total the list elements:

```
total = sum(numbers)          # total is 188
```

# Section Problem: Fruit List

# Section Problem: Fruit List

- Create a list of these fruits: apple, banana, orange, grape, pineapple
- Print the length of the created list
- Add mango to the end of the list
- Print the updated list. Try two different approaches:
  - Print the entire list on one line
  - Print each fruit on its own line

# That's all, folks!

Next up:  Dictionaries!

# Bonus Slides

File I/O

# Accessing data from files (Slide 1 of 3)

- Python can read and write plain text files
- Usually these have the filename extension **`.txt`**
- **`.py`**, **`.html`**, **`.cpp`**, or other source code files are also typically plain text
- Sometimes plain text files lack an extension altogether
- Notepad, TextEdit,  BBEdit, and other text editors can create plain text files
- Microsoft Word and other word processors can export files as plain text
- Opening a text file in  Python can be a security risk
- Open only files from a trusted and verified source!
    - Anyone can send a file purporting to be from your boss, your bank, your family member ...

# Accessing data from files (Slide 2 of 3)

- To open a file for reading or writing, use the **`with open`** statement:

```
with open (path/to/file, mode) as fh:
    # indented block to read or write the file
```

- **`fh`** is whatever variable name you want to use for the file
  - **`fh`** is common, as it's short for **`file handle`**
  - **`infile`** is also common for files to be read
  - **`outfile`** is also common for files to be written
- *`path/to/file`* is a **`str`** specifying the location of the file on the system
  - The path can be relative or absolute
  - It is often a constant or received as input from the user

# Accessing data from files (Slide 3 of 3)

- To open a file for reading or writing, use the **`with open`** statement:

```
with open (path/to/file, mode) as fh:
    # indented block to read or write the file
```

- *`mode`* is a **`str`** specifiying one of three options:
  - **`'r'`** for read. This is the default and can be omitted
  - **`'w'`** for write
    - If the file does not exist, it will be created
    - Watch out! If the file already exists, it will be overwritten!
  - **`'a'`** for append
    - If the file exists, the new data will be added to the end
    - If the file does not exist, it will be created

# Reading from a file

- Python reads the file one line at a time, each line as a **`str`**
- It's usually efficient to loop over all the lines and add them to a list
- Each element of the list is one line of the file as a **`str`**
- Sample code:

```
lines = []
with open ('file_to_read.txt', 'r') as infile:
    for line in infile:
        lines.append(line.strip())
```

- **`strip()`** removes whitespace from the beginning and end of the lines, such as tabs **`'\t'`** or newlines **`'\n'`**

# Writing to a file

- Python can write strings out to a file
- Remember that the `'w'` option will overwrite the file if it exists!
- Say you have a list of names: `['Jane', 'John', 'June', 'Joe']`
- The following code will write the names out to the file, one name per line:

```
with open ('file_to_write.txt', 'w') as outfile:
    for name in names:
        outfile.write(f'{name}\n')
        # or outfile.write(name + '\n')
```

- Note that unlike `print()`, you must explicitly supply `'\n'` to `write()`